前端页面脚本扩展

变更记录

产品版本	更新内容	更新日期
V7.0.1	发布了前端页面脚本功能，可以编写javascript代码，从而快速实现特定的业务控制逻辑等需求	2024-10-24
V7.0.2	1、新增获取服务端数据API "fetchData"，打通页面脚本与服务端插件通信（具体案例见章节九） 2、新增自定义控件可视化绑定事件"onCustomMsgEvent"，支持自定义控件内部向页面脚本中发送消息（具体介绍见章节四） 3、新增自定义控件预置事件消息"init"（具体介绍见章节四） 4、脚本编辑器内容保存，支持快捷键"Ctrl + S 或 ⌘ + S"	2024-11-28
V7.0.3	1、按钮/工具栏/页签工具栏/高级面板工具栏等支持标准点击事件，包含分录的场景（具体介绍见章节四） 2、树控件支持节点自定义渲染（具体介绍见章节四和章节九案例） 3、表格控件支持单元格自定义渲染（具体介绍见章节四和章节九案例） 4、支持开启全局变量"show_ps_init"，脚本初始化时输出固定log信息，方便调试（具体介绍见章节八）	2024-12-10
V7.0.4	1、表格(单据体)控件新增若干脚本事件与功能接口（详见章节四"表格"部分内容） 2、树控件新增若干脚本事件与功能接口（详见章节四"树"部分内容）	2024-12-24
V7.0.7	1、运行时脚本支持单独接口请求（接口名称"getPageJs.do"）、实现前后端性能优化	2025-3-13
V7.0.11	1、表格自定义渲染接口setCellRender其props对象新增参数originValue，用于暴露单元格真实数值 2、支持前端页面脚本的单据类型基本覆盖到所有页面（包括标准列表等），满足不同用户在不同类型单据做脚本开发的需要	2025-5-9

V7.0.13	1、修复了标准列表的“列表表单模板”属性值不是"bos_list、bos_templatetreelist"时，导致在列表页面编写的页面JS脚本无法加载，不生效的问题 2、修复了在列表的"列表表单模版"单据里(实际为动态表单)里编写的页面JS脚本，在列表页面渲染时不生效的问题	2025-6-10
V8.0.1	1、修复了移动端单据被内嵌在容器中作为子单据使用时，其页面脚本不生效的问题	2025-10-24
V8.0.3	1、修复了页面脚本setValue接口在字段被锁定状态下会赋值失败的问题	2025-12-3

一、功能概述

通过前端页面脚本扩展功能，用户可以直接编写javascript代码，从而快速实现特定的业务控制逻辑
易于使用，对现有业务逻辑进行扩展操作简单（前提掌握脚本开发）
部署方便，脚本跟随元数据一起导入导出、补丁升级
支持内置API和组件事件等，增强了前端页面定制化开发能力
如想快速查看相关定制效果，可直接查看章节九"案例展示"阅读对应内容

二、适用场景

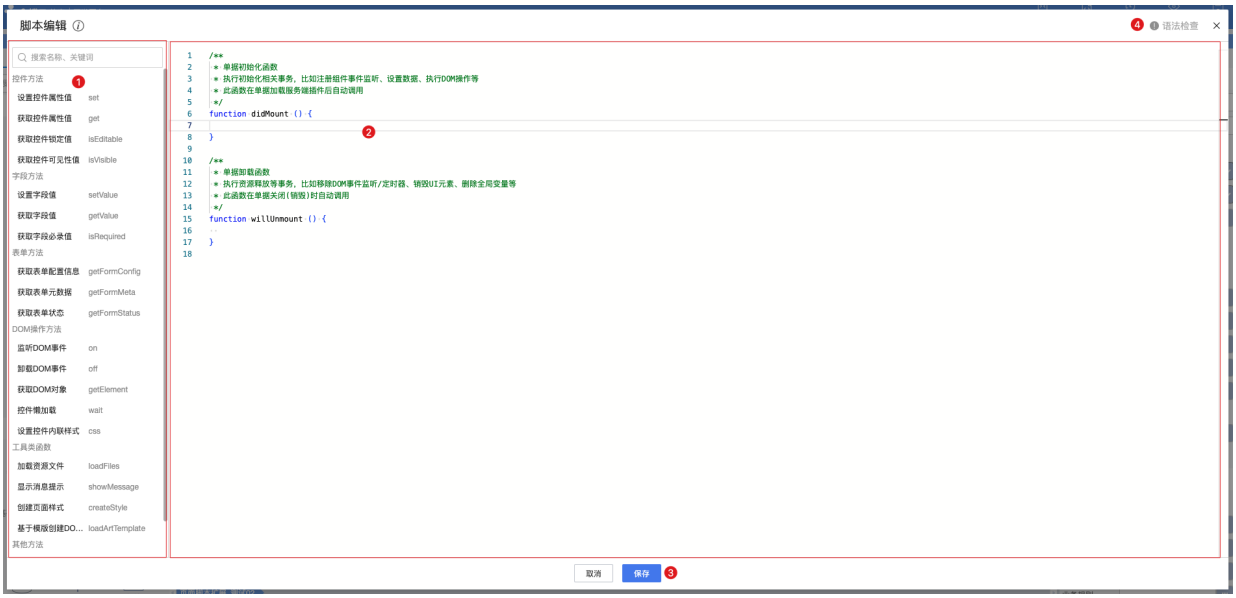
以下场景可以考虑使用页面脚本扩展功能来实现：
1、想在前端页面快速实现一些简单的业务控制逻辑
2、依据页面数据状态等需要对某些UI元素做定制开发（比如修改内容展示格式，样式以及动画等）
3、开发的自定义控件其渲染数据依赖页面上字段等数据的变化，不想通过服务端插件实现
4、其他基于web页面轻量化JS脚本想实现的功能等
注意：
1、标准业务与二开项目等都可使用（有做权限控制）
2、**脚本在设计器中可进行编辑或查看，且考虑到web页面脚本的安全性等，不建议将重要或复杂的业务逻辑放在脚本中实现**
3、**如果想开发一个独立控件且需要与服务端插件通信，同时希望在不同单据页面可复用，则推荐使用自定义控件做开发**

三、功能入口与界面介绍

该页面脚本功能分设计时与运行时，设计时可以编写/查看、启用/禁用脚本，运行时执行脚本。

1、设计时

脚本编辑器



（1）API列表

- 基于这些内置API可快速实现特定功能。双击API名称快速插入代码片段、鼠标悬浮右侧"!"号上展示对应API详细内容

（2）脚本代码编辑区域

- 在此编辑JS脚本。支持录入功能接口或控件key等场景，自动提示并展示下拉选项列表

（3）脚本"保存"按钮

- 点击直接保存脚本（**脚本保存不依赖设计器的"保存"操作，即无需再点击设计器右上角保存按钮**）

（4）脚本语法检查

- 检查脚本内容是否存在语法错误

（5）快速保存脚本

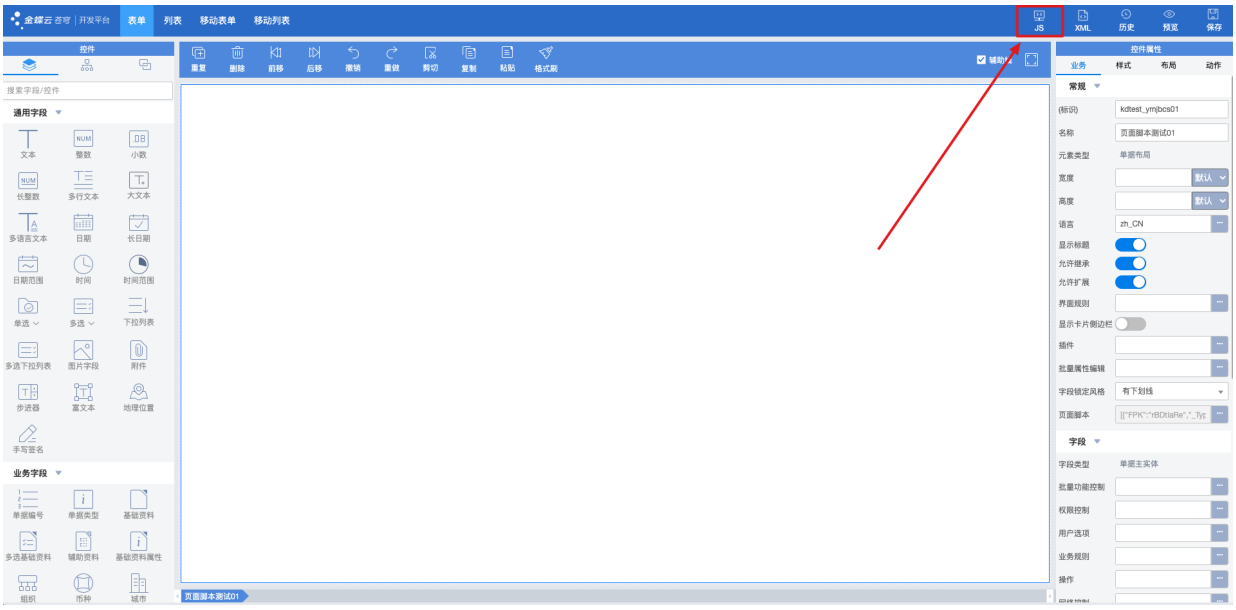
- 在脚本编辑界面通过快捷键"**Ctrl + S 或 ⌘ + S**"可快速保存脚本，无需关闭界面

脚本编辑器入口

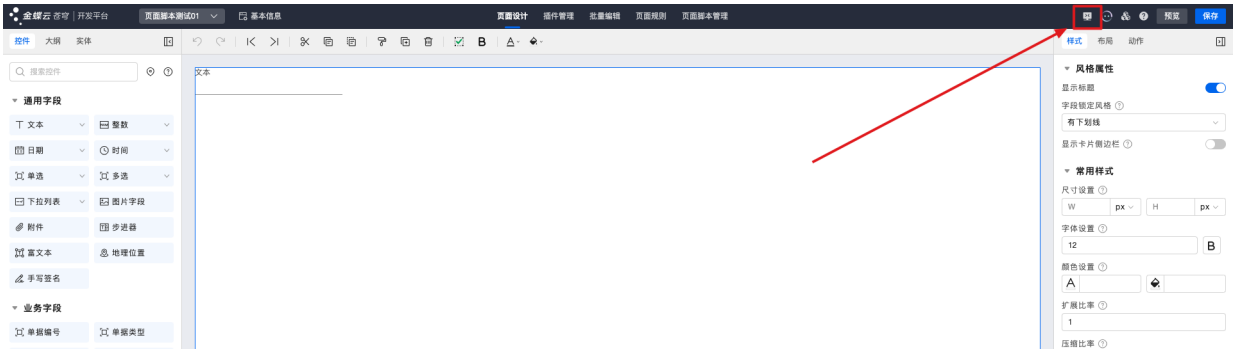
脚本编辑和打开脚本编辑器的入口都在表单设计器中。

总入口

点击该入口，会打开脚本编辑器并加载对应脚本内容。
【旧版 - 表单设计器】



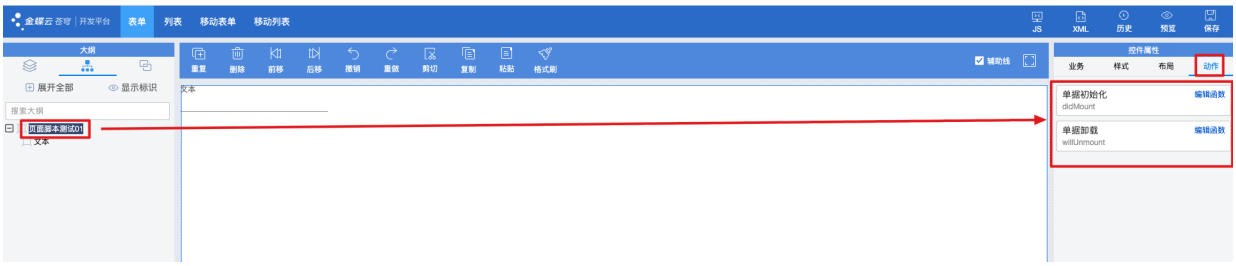
【新版 - 表单设计器】



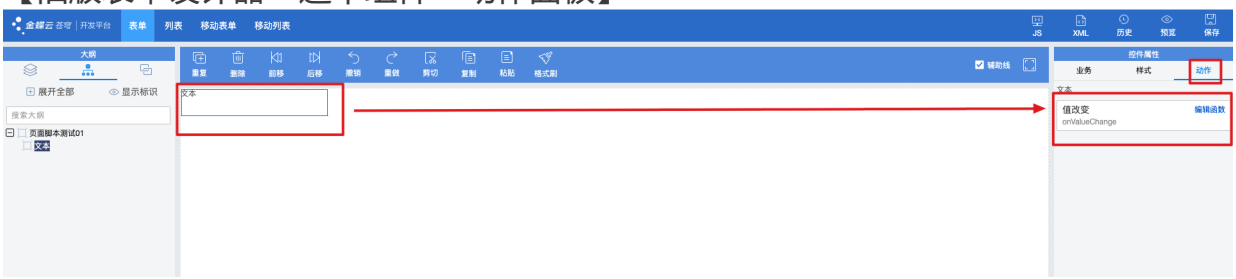
页面/组件事件绑定入口

点击该入口，会打开脚本编辑器并自动生成对应组件/页面的事件绑定代码（如果已绑定过事件则会自动定位到对应代码行位置）。

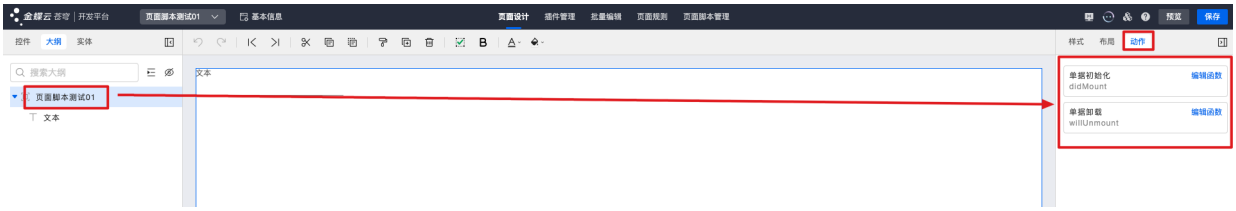
【旧版表单设计器 - 选中单据根节点 - 动作面板】



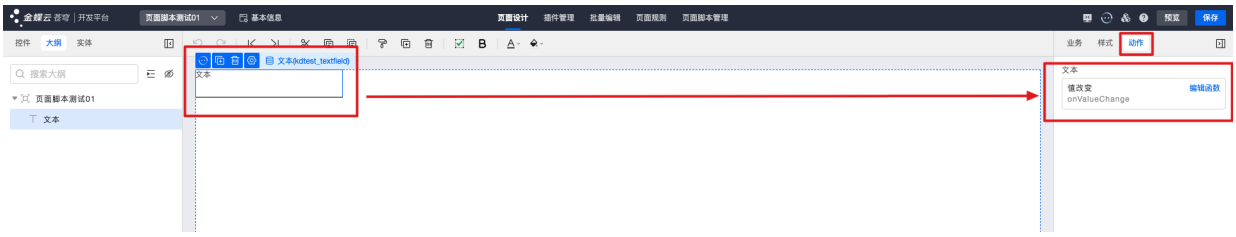
【旧版表单设计器 - 选中组件 - 动作面板】



【新版表单设计器 - 选中单据根节点 - 动作面板】



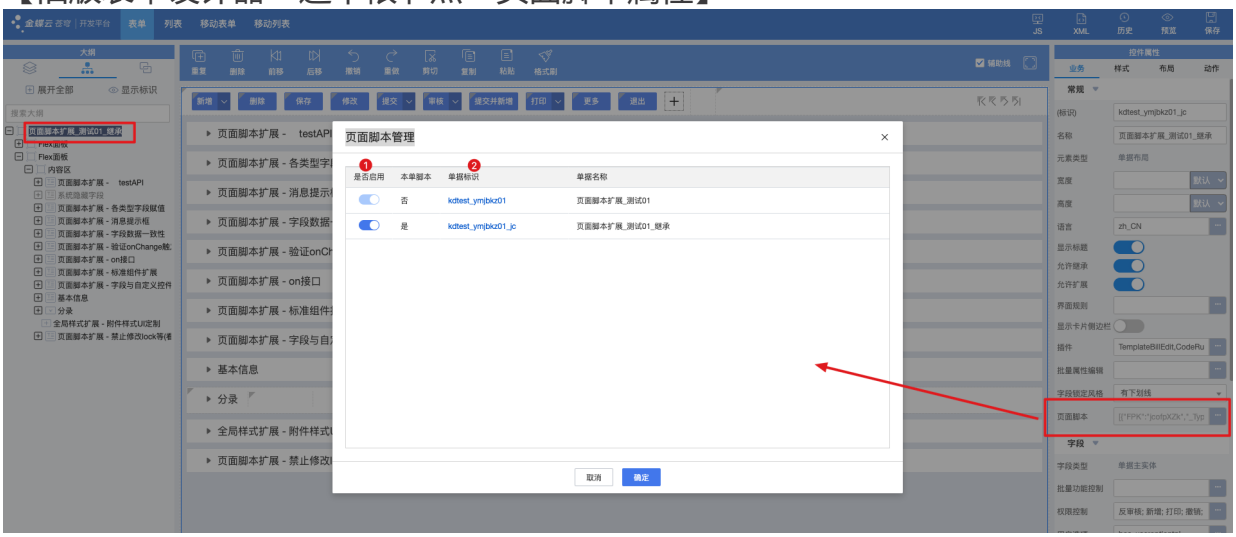
【新版表单设计器 - 选中组件 - 动作面板】



脚本管理界面

脚本管理界面可查看单据自身及其父页面的脚本内容，同时如有编辑权限可启用/禁用当前页面脚本

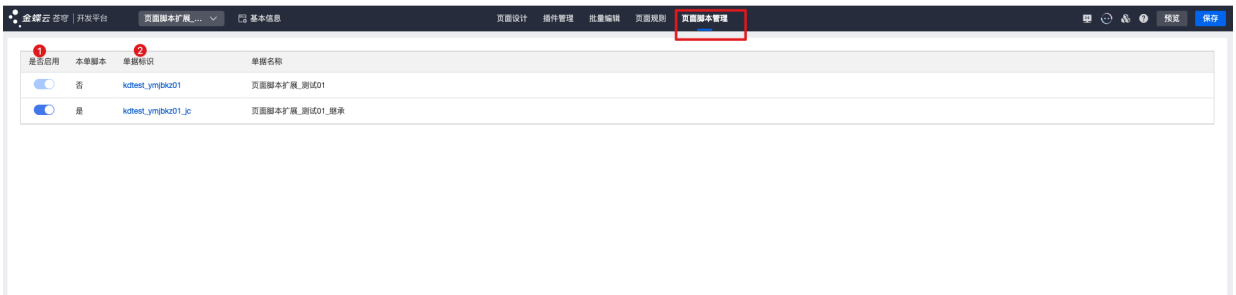
【旧版表单设计器 - 选中根节点 - 页面脚本属性】



（1）启用/禁用脚本

（2）点击"表单标识"列超链接，可查看对应脚本内容

【新版表单设计器 - 页面脚本管理页签】



权限控制

权限控制，与Java插件管理一致，设计器中相同开发商才允许编辑。

2、运行时

执行脚本

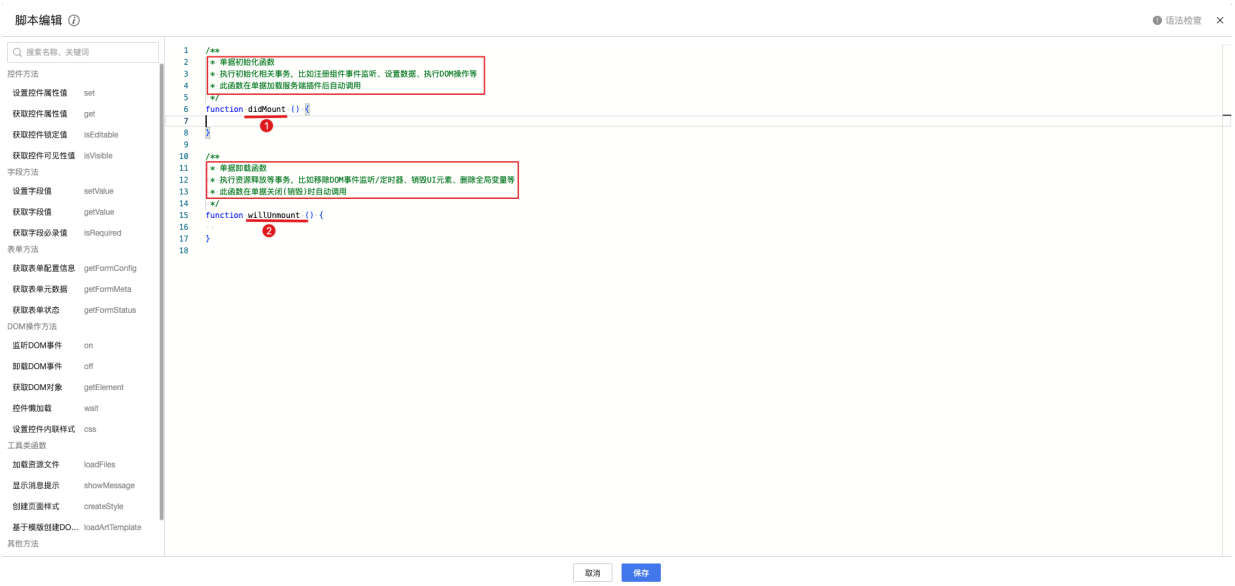
- 运行时打开一个单据，单据对应脚本就会执行。
- 当存在多份脚本时（即父页面也有脚本的情况），脚本执行顺序为：自顶向下，优先执行父页面脚本。

四、脚本编写规范与语法支持

脚本编写使用javascript，内置了一些脚本对象、API和标准事件等。

一个单据页面自身只会有一份JS脚本，同时支持通过单据继承/扩展/复制等方式来继承父页面的脚本。

脚本生命周期



脚本编写主要在两个大的生命周期函数中进行，分别为：didMount、willUnmount

didMount

- 单据初始化函数
- 执行初始化相关事务，比如注册组件事件监听、设置数据、执行DOM操作等
- 此函数在单据加载服务端插件后自动调用

注意：
1、一般脚本代码的主要实现逻辑都建议在该函数中编写实现
2、该函数在页面业务数据加载完成之后执行（即"loaddata"请求）

willUnmount

- 单据卸载函数
- 执行资源释放等事务，比如移除DOM事件监听/定时器、销毁UI元素、删除全局变量等
- 此函数在单据关闭(销毁)时自动调用

注意：该函数在页面关闭时执行

事件与功能接口

支持范围：包括单据页面和各类控件，PC和移动端
事件：用于在页面脚本中做监听，特定操作或数据状态下会触发，在其回调方法中实现自己的业务逻辑
功能接口：控件对象对外暴露的功能接口，通过在页面脚本中调用该接口可使得对应控件实现特定的业务逻辑

单据页面

didMount

单据初始化事件，设计器中选中单据根节点，右侧"**动作**"面板可看到对应事件绑定入口（**新增脚本默认自动生成**）
示例：

```
/**
 * 单据初始化函数
 * 执行初始化相关事务，比如注册组件事件监听、设置数据、执行DOM操作等
 * 此函数在单据加载服务端插件后自动调用
 */
function didMount () {

}
```

willUnmount

单据卸载事件，设计器中选中单据根节点，右侧"**动作**"面板可看到对应事件绑定入口（**新增脚本会默认自动生成**）
示例：

```
/**
 * 单据卸载函数
 * 执行资源释放等事务，比如移除DOM事件监听/定时器、销毁UI元素、删除全局变量等
 * 此函数在单据关闭(销毁)时自动调用
 */
function willUnmount () {

}
```

【可视化绑定入口】

字段

onValueChange

字段值更改事件，设计器中选中任意字段类型组件，右侧"**动作**"面板可看到对应事件绑定入口

- 字段值更改事件，在内容发生改变且字段输入框失去焦点时触发
- 后续会根据规划与需要开放更多组件的标准事件(比如按钮点击等)

示例：

```
// 事件参数结构说明：data: {key, newValue, oldValue}
this.$('字段标识').onValueChange((data) => {
    // todo something
})
```

【可视化绑定入口】

自定义控件

onCustomMsgEvent

自定义控件的自定义消息订阅事件，设计器中选中自定义控件，右侧"**动作**"面板可看到对应事件绑定入口

- 在自定义控件内部调用"**this.model.triggerCustomMsgEvent(xx, xx)"接口触发该事件（即实现自定义控件内部向页面脚本中发送消息，接口调用时机和参数由开发者自定义）**
- 基于该事件可以按需从自定义控件内部向页面脚本中发送消息，且消息内容可自定义
- 支持在卡片分录中使用（如果在卡片分录中使用，则事件参数多一个rowIndex参数，代表所在分录行）

示例：

【页面脚本 - 事件绑定代码】

```
//页面脚本中事件绑定代码。参数结构说明：data: {type, args}，type和args参数都不一定存在，看自定义控件内部传递的参数
this.$('自定义控件标识').onCustomMsgEvent((data) => {

})
```

【自定义控件内部 - 发送消息/触发事件代码】

```
//调用示例
this.model.triggerCustomMsgEvent('dataChange', {name: '名称1', code: '888'}); //调用：接口方式一
this.model.triggerCustomMsgEvent({type: 'dataChange', name: '名称1', code: '888'}); //调用：接口方式二
```

【可视化绑定入口】

预置"init"事件消息

如果页面脚本中绑定了onCustomMsgEvent事件，自定义控件加载执行完成生命周期函数"init"之后会自动触发自定义控件事件消息

- 此时事件参数内容为 {"key":"控件标识","args":{"type":"__init__"}}可基于此事件消息在页面脚本中处理自定义控件相关附加逻辑（**注意**：此时自定义控件DOM结构不一定已经完全加载完成，依赖具体实现）

示例：
【页面脚本 - 事件绑定代码】

```
1. //页面脚本中事件绑定代码。参数结构说明：data: {"key":"控件标识","args":{"type":"__init__"}},type和args参数都不一定存在，看自定义控件内部传递的参数
2. this.$('自定义控件标识').onCustomMsgEvent((data) => {
3.     if(data.args.type === "__init__"){
4.         //todo something
5.     }
6. })
```

按钮

onClick

按钮单击事件，设计器中选中按钮控件，右侧"**动作**"面板可看到对应事件绑定入口

- 按钮被锁定，则该事件不会触发
- 支持在卡片分录中使用（如果在卡片分录中使用，则事件参数多一个rowIndex参数，代表所在分录行）

示例：
【页面脚本 - 事件绑定代码】

```
1. //页面脚本中事件绑定代码（参数data包含控件标识和绑定的操作代码）
2.   this.$('按钮标识').onClick((data) => {
3.
4.   })
```

【可视化绑定入口】

工具栏

onItemClick

工具栏单击事件，设计器中选中整个工具栏控件，右侧"**动作**"面板可看到对应事件绑定入口

- 点击的工具栏按钮被锁定，则该事件不会触发
- 动态增加的工具栏按钮也可以通过该事件监听到
- 事件绑定是监听整个工具栏里按钮的点击，点击的一级或二级等按钮标识和对应操作代码，都包含在事件参数中

示例：
【页面脚本 - 事件绑定代码】

```
//页面脚本中事件绑定代码
this.$('工具栏标识').onItemClick((data) => {

})
```

【可视化绑定入口】

高级面板工具栏

onItemClick

高级面板工具栏的单击事件，设计器中选中整个高级面板的工具栏控件，右侧"**动作**"面板可看到对应事件绑定入口

- 工具栏按钮被锁定，则该事件不会触发
- 事件绑定是监听整个工具栏里按钮的点击，点击的一级或二级等按钮标识和对应操作代码，都包含在事件参数中

示例：
【页面脚本 - 事件绑定代码】

```
//页面脚本中事件绑定代码
this.$('高级面板工具栏标识').onItemClick((data) => {
```

```
3.
4. })
```

【可视化绑定入口】

页签操作

onItemClick

页签控件中每个子页签的“页签操作”按钮点击事件，设计器中选中整个页签控件，右侧"**动作**"面板可看到对应事件绑定入口

- 点击的页签操作按钮被锁定，则该事件不会触发
- 事件绑定是监听整个页签操作的点击，点击的操作按钮标识、子页签标识和对应操作代码，都包含在事件参数中

示例：
【页面脚本 - 事件绑定代码】

```
1. //页面脚本中事件绑定代码
2. this.$('页签控件标识').onItemClick((data) => {
3.
4. })
```

【可视化绑定入口】

树

自定义渲染

setTreeItemRender

功能接口，支持树节点自定义渲染或扩展
接口说明

```
1. // 接口名称
2. setTreeItemRender
3. // 接口参数，自定义渲染器即一个render函数（该函数中的参数中包含节点等相关信息）
```

```
TreeItemRender
// 接口调用
this.$('控件标识').setTreeItemRender(TreeItemRender)
```

具体案例见章节九 “案例展示 - 树控件节点自定义渲染或扩展”
注意：
1、多脚本（如继承单据）渲染器生效规则：同一个树控件，子页面脚本覆盖父页面脚本的自定义渲染器
2、Render函数最终返回的是一个react对象，支持使用react框架语法编写（目前支持react框架版本16.8）

事件

onInit

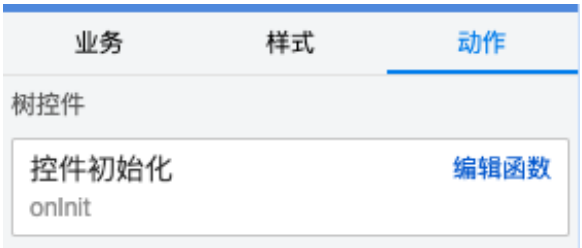
树控件"**控件初始化事件**"，设计器中选中树组件，右侧"**动作**"面板可看到对应事件绑定入口

- 树控件相对复杂，跟随单据加载进行初始化需要一定时间，如果要在"didMount"中准确获取该控件加载完成的时机，可使用该事件

示例：

```
// 树控件-控件初始化
this.$('树控件标识').onInit().then(() => {

})
```

【可视化绑定入口】



onTreeNodeClick

树节点"**单击事件**"，设计器中选中树组件，右侧"**动作**"面板可看到对应事件绑定入口
示例：

```
// 树控件-单击
this.$('树控件标识').onTreeNodeClick((data) => {

})
```

【可视化绑定入口】

onTreeNodeDoubleClick

树节点"**双击事件**"，设计器中选中树组件，右侧"**动作**"面板可看到对应事件绑定入口
示例：

```
// 树控件-双击
this.$('树控件标识').onTreeNodeDoubleClick((data) => {

})
```

【可视化绑定入口】

onTreeNodeCheck

树控件节点"**勾选/取消勾选事件**"，设计器中选中树组件，右侧"**动作**"面板可看到对应事件绑定入口
示例：

```
// 树控件-选中节点
this.$('树控件标识').onTreeNodeCheck((data) => {

})
```

【可视化绑定入口】

功能接口

expand

设置节点展开
接口格式

```
1. this.$('树控件标识').expand(节点id)
```

调用示例

```
1. this.$('treeap').expand('c1_c2')
```

collapse

折叠指定节点
接口格式

```
1. this.$('树控件标识').collapse(节点id)
```

调用示例

```
1. this.$('treeap').collapse('c1_c2')
```

checkNodes

勾选一批节点
接口格式

```
1. this.$('树控件标识').checkNodes([节点id])
```

调用示例

```
1. this.$('treeap').checkNodes(['c1_c2','c1_c3'])
```

uncheckNodes

取消一批节点勾选

接口格式

```
1. this.$('树控件标识').uncheckNodes([节点id])
```

调用示例

```
1. this.$('treeap').uncheckNodes(['c1_c2','c1_c3'])
```

getParent

获取指定节点的父节点数据
接口格式

```
1. this.$('树控件标识').getParent(节点id)
```

调用示例

```
1. this.$('treeap').getParent('c1_c2')
```

getAllParent

获取指定节点的所有父节点数据（到根节点为止）
接口格式

```
1. this.$('树控件标识').getAllParent(节点id)
```

调用示例

```
1. this.$('treeap').getAllParent('c1_c2')
```

getNode

获取特定节点数据
接口格式

```
1. this.$('树控件标识').getNode(节点id)
```

调用示例

```
1. this.$('treeap').getNode('c1_c2')
```

getTreeData

获取树所有节点数据
接口格式

```
1. this.$('树控件标识').getTreeData()
```

调用示例

```
1. this.$('treeap').getTreeData()
```

getTreeState

获取树形控件当前状态，包括焦点节点、已勾选的节点等信息
接口格式

```
this.$('树控件标识').getTreeState()
```

调用示例

```
this.$('treeap').getTreeState()
```

表格

自定义渲染

支持范围：单据体（查看态和编辑态）、单据列表、标准列表

setCellRender

功能接口，用于表格单元格（**单元格查看时，即非编辑态**）自定义渲染或扩展

- 支持重新渲染整个单元格
- 支持基于现有单元格渲染扩展新的UI元素

接口说明

```
// 接口名称（自定义渲染表格单元格）
setCellRender
// 接口参数，自定义渲染器即一个render函数（该函数的参数中包含单元格等相关信息）或{}对象
CustomCellRender
// 使用方式1：自定义渲染所有单元格
this.$('表格标识').setCellRender(CustomCellRender)
// 使用方式2：按特定列自定义渲染单元格
this.$('表格标识').setCellRender({
    列id1: Render渲染函数1,
    列id2: Render渲染函数2
})
```

具体案例见章节九 “案例展示 - 表格单元格自定义渲染或扩展” 部分内容
注意：
1、Render函数“CustomCellRender”的参数props中有新增接口“isCellLock”，用于获取单元格是否锁定，根据该状态值渲染查看态UI效果
2、单元格锁定状态下，是无法进入编辑态的，所以编辑器的自定义渲染不用处理锁定状态渲染
3、多脚本（如继承单据）渲染器生效规则：同一个表格控件的同一列，子页面脚本覆盖父页面脚本的自定义渲染器
4、**【重要！】**使用该接口基于单元格渲染器Render扩展单元格时，从props中获取的“value”属性为**格式化后的值（用于在单元格中显示，非原始值；原始值要从props中的属性“originValue”获取）**，该值内容格式随“国际化”设置而变化，比如数字类型字段其值为"0.7"，设置国际化之后可能变为",7"，所以对value属性值进行处理时请谨慎，以防出现程序异常等情况

```
<Render {...{ rowIndex, renderProps, record, value }} />
```

setCellEditor

功能接口，用于表格单元格（**单元格编辑时，即编辑态**）自定义渲染或扩展

- 支持重新渲染整个单元格
- 支持基于现有单元格渲染扩展新的UI元素

接口说明

```
//接口名称
setCellEditor
//接口参数，自定义渲染器即即一个render函数（该函数的参数中包含单元格等相关信息）或{}对象
CustomCellEditor
//使用方式：按特定列自定义渲染单元格编辑态
this.$('表格标识').setCellEditor({
    列id1: Render渲染函数1,
    列id2: Render渲染函数2
})
```

具体案例见章节九 “案例展示 - 表格单元格自定义渲染或扩展” 部分内容
注意：
1、如果是用户二开接管了表格编辑态，则自定义单元格编辑在编辑完成后还要将最新数据通过单元格值更新udateValue请求同步回服务端。
2、表格查看态渲染，需要关注单元格锁定状态，其渲染要自己处理锁定视觉效果。
3、单元格锁定状态下是无法进入编辑状态的，所以编辑态的单元格自定义渲染不需要关注锁定状态。
4、Render函数最终返回的是一个react对象，支持使用react框架语法编写（目前支持react框架版本16.8）

事件

onInit

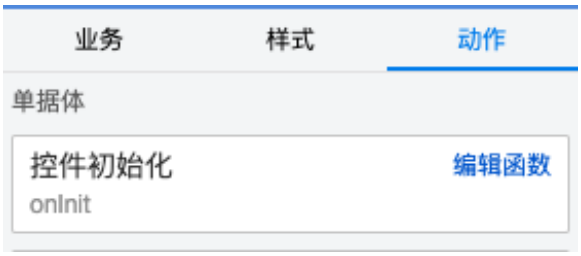
表格控件"**控件初始化事件**"，设计器中选中表格组件，右侧"**动作**"面板可看到对应事件绑定入口

- 表格控件（单据体）相对复杂，跟随单据加载进行初始化需要一定时间，如果要在"didMount"中准确获取该控件加载完成的时机，可使用该事件

示例：

```
// 表格控件-控件初始化
this.$('表格控件标识').onInit().then(() => {

})
```

【可视化绑定入口】



onTableRowClick

表格"**行单击事件**"，设计器中选中表格组件，右侧"**动作**"面板可看到对应事件绑定入口
示例：

```
// 单据体-行单击
this.$('表格控件标识').onTableRowClick((data) => {

})
```

【可视化绑定入口】

onTableRowDoubleClick

表格"**行双击事件**"，设计器中选中表格组件，右侧"**动作**"面板可看到对应事件绑定入口
示例：

```
// 单据体-行双击
this.$('表格控件标识').onTableRowDoubleClick((data) => {

})
```

【可视化绑定入口】

onCellValueChange

表格"**单元格值改变**"，设计器中选中表格组件，右侧"**动作**"面板可看到对应事件绑定入口
示例：

```
// 单据体-单元格值改变
this.$('表格控件标识').onCellValueChange((data) => {

})
```

【可视化绑定入口】

onSelect

表格"**勾选事件**"，设计器中选中表格组件，右侧"**动作**"面板可看到对应事件绑定入口
示例：

```
// 单据体-勾选
this.$('表格控件标识').onSelect((data) => {

})
```

【可视化绑定入口】

onUnSelect

表格"**取消勾选事件**"，设计器中选中表格组件，右侧"**动作**"面板可看到对应事件绑定入口
示例：

```
// 单据体-取消勾选
this.$('表格控件标识').onUnSelect((data) => {

})
```

【可视化绑定入口】

onSelectAll

表格"**全选事件**"，设计器中选中表格组件，右侧"**动作**"面板可看到对应事件绑定入口
示例：

```
// 单据体-全选
this.$('表格控件标识').onSelectAll((data) => {

})
```

【可视化绑定入口】

onUnSelectAll

表格"**取消全选事件**"，设计器中选中表格组件，右侧"**动作**"面板可看到对应事件绑定入口
示例：

```
// 单据体-取消全选
this.$('表格控件标识').onUnSelectAll((data) => {

})
```

【可视化绑定入口】

功能接口

setCellValue

设置单元格值
接口格式

```
this.$('表格控件标识'). setCellValue([{k: '列标识', r: 行号,v: 单元格值}])
```

调用示例

```
this.$('entryentity').setCellValue([{k: 'kdtest_textfield', r: 8,v: '888'}])
```

setCellStyle

设置单元格样式
接口格式

```
// s：样式对象、bc代表背景色属性，还有前景色属性fc，字体大小fs等
this.$('表格控件标识'). setCellStyle([{k: '列标识', r: 行号,s: { bc: 颜色值 }}])
```

调用示例

```
this.$('entryentity').setCellStyle([{k: 'kdtest_textfield', r: 8,s: { bc: '#ff0000' }}])
```

setRowStyle

设置行样式
接口格式

```
//r：行号，支持批量、s：样式对象、bc代表背景色属性，还有前景色属性fc，字体大小fs等
this.$('表格控件标识'). setRowStyle([{r: [行号],s: { bc: 颜色值 }}])
```

调用示例

```
this.$('entryentity').setRowStyle([{r: [8, 1],s: { bc: '#ff0000' }}])
```

setSelectRows

设置勾选行
接口格式

```
//单个勾选
this.$('表格控件标识'). setSelectRows(行号)
//批量勾选
this.$('表格控件标识'). setSelectRows([行号])
```

调用示例

```
this.$('entryentity').setSelectRows(1)
this.$('entryentity').setSelectRows([1,3,5]) //批量勾选
```

getRowData

获取表格行数据
接口格式

```
this.$('表格控件标识').getRowData(行号)
```

调用示例

```
this.$('entryentity').getRowData(1)
```

getGridData

获取表格所有数据
接口格式

```
this.$('表格控件标识').getGridData()
```

调用示例

```
this.$('entryentity').getGridData()
```

getGridState

获取选中行状态数据（包括焦点行，选中行等）
接口格式

```
this.$('表格控件标识').getGridState()
```

调用示例

```
this.$('entryentity').getGridState()
```

getFocusedCell

获取焦点单元格
接口格式

```
this.$('表格控件标识').getFocusedCell()
```

调用示例

```
this.$('entryentity').getFocusedCell()
```

内置对象

"**this**"：页面脚本上下文对象，用于访问页面内所有控件实例和获取页面自身相关数据。

- 该对象一般在didMount或willUnmount中使用，使用时注意要保证其指向正确（特别是在嵌套函数中），否则脚本会报错，无法正确访问相关对象或接口

示例：

```
 function didMount() {
     // this指向页面脚本上下文
     this.xxx

     //错误写法
     this.$('控件标识').on('click',function(e){
         //此时this不指向页面脚本上下文，js会报错
         this.$('控件标识').set('Name','title')
     })

     //正确写法（使用箭头函数，自动绑定上下文）
     this.$('控件标识').on('click',(e)=>{
         //此时this指向页面脚本上下文,js正常执行
         this.$('控件标识').set('Name','title')
     })
 }

 function willUnmount() {
```

```
        // this指向页面脚本上下文
        this.xxx
    }

    // 此时this不指向页面脚本上下文（指向window对象）
    this.xxx
```

"this.$('控件标识')"：用于获取控件实例，使用该实例可调用控件相关功能接口（包括通用控件方法、字段方法等）
示例：

```
// 获取控件实例对象
this.$('控件标识')

//调用控件实例对象功能接口
this.$('控件标识').set('Name', 'title')
```

"export"关键字

如果将某些逻辑封装为独立的函数，且函数中用到了脚本内置对象或API等，为了在函数中能正常访问这些对象或API等，则需要为函数增加**export**修饰关键字，同时函数调用方式为：**this.独立函数名称()**。
示例:

```
    // 初始化
   function didMount() {
        this.testMethod() //调用函数
   }

// 独立函数写法
export function testMethod(str) {
    this.$('控件标识').setValue('111')
}
```

与自定义控件通信

标准字段值变化要通知自定义控件，需要调用invoke接口。
具体示例代码见 "**案例展示 - 标准字段（页面脚本）与自定义控件通信**"部分内容。

五、API介绍

页面脚本编写，预制了部分API，提升代码编写效率与使用体验。
具体API及其分组如下：

1、通用控件方法

set

设置控件属性值（支持所有控件，属性名称可在设计器右侧属性面板查看）
格式：

```
this.$('控件标识').set('属性名', '属性值')
```

示例：

```
this.$('textfield1').set('Name', 'title')
```

get

获取控件的属性值（属性名称可在设计器右侧属性面板查看）
格式：

```
this.$('控件标识').get('属性名')
```

示例：

```
this.$('textfield1').get('Name')
```

isEditable

获取控件锁定性，即控件是否可编辑
格式：

```
this.$('控件标识').isEditable()
```

示例：

```
this.$('textfield1').isEditable()
```

isVisible

获取控件可见性，即是否被隐藏
格式：

```
this.$('控件标识').isVisible()
```

示例：

```
this.$(textfield1').isVisible()
```

2、字段方法

setValue

设置字段值
格式：

```
this.$('控件标识').setValue('值内容')
```

示例：

```
1. this.$('textfield1').setValue('123')
```

getValue

获取字段值
格式：

```
1. this.$('控件标识').getValue()
```

示例：

```
1. this.$('textfield1').getValue()
```

注：基础资料和多语言类型字段通过该接口获取到的数据是一个不可变对象，此时可通过**toJS接口**将其转为一般对象使用

```
1. let val = this.$('basedatafield').getValue() // "basedatafield"是一个基础资料类型字段的标识
2. if(val && val.toJS){ //不可变对象有toJS接口
3.     val = val.toJS()
4. }
```

isRequired

获取字段必录属性，表明该字段是否是必须录入的
格式：

```
1. this.$('控件标识').isRequired()
```

示例：

```
1. this.$('textfield1').isRequired()
```

3、表单方法

getFormConfig

获取当前表单的配置信息，包含表单相关配置项
示例：

```
1. this.getFormConfig()
```

getFormMeta

获取当表表单的元数据，包含所有组件的布局等描述信息
示例：

```
1. this.getFormMeta()
```

getFormStatus

获取当前表单的单据状态，其返回数据含义：0/新增、1/修改、2/查看、4/提交、5/审核
示例：

```
1. this.getFormStatus()
```

fetchData

获取服务端数据，向服务端发送请求，可自定义请求参数：（methodName/请求方法名称、param/请求参数）；服务端返回数据使用全局指令：setPageJSData
格式：

```
1. this.fetchData(methodName, param).then((result)=>{})
```

示例：

```
1. this.fetchData('getUserInfo', {userName:'xiaoming',
   userCode:'123'}).then((result)=>{
2. })
```

服务端响应说明：
1、前端页面直接调用fetchData API接口发送网络请求，服务端插件通过重载**自定义事件customEvent**来接收请求消息（插件即为脚本所在页面的插件）。
2、发送到服务端请求使用了固定自定义事件key："**__clientRequest__**"，以此可以判断是不是页面脚本的请求
具体使用案例见本文档："案例展示 - 页面脚本与服务端插件通信" 章节内容

4、DOM操作方法

注意：DOM相关操作如果依赖控件对应DOM结构上的某个子元素，请谨慎使用，后续版本迭代升级无法保证DOM结构不会变化。

on

用于为DOM元素添加事件监听，接受三个参数：事件名称（HTML原生事件，可同时监听多个，用空格分隔）、子元素选择器 【可选】、事件回调函数
格式：

```
1. this.$('控件标识').on('html事件名称', selector[可选], 事件回调函数)
```

示例：

```
1. const handler=event=>{...}
2. this.$('textfield1').on('mouseenter mouseleave', handler)
```

注意：如果需要监听控件中某个子元素的事件，就需要用到子元素选择器selector，但其中获取焦点和失去焦点事件"focus、blur"的写法特殊（事件名称要替换一下"focus -> focusin、blur -> focusout"），具体参考如下：

```
1. //写法推荐
```

```
//获取焦点
const handler=event=>{...}
this.$('textfield1').on('focusin', 'input', handler)
//失去焦点
this.$('textfield1').on('focusout', 'input', handler)

//不生效写法
this.$('textfield1').on('focus', 'input', handler)
this.$('textfield1').on('blur', 'input', handler)
```

off

用于移除DOM元素的事件监听，可移除通过'on'接口监听的所有或特定事件
格式：

```
this.$('控件标识').off('html事件名称', 事件回调函数)
```

示例：

```
const handler=event=>{...}
this.$('textfield1').off('mouseenter mouseleave',handler)
```

getElement

获取控件对应的DOM对象
格式：

```
this.$('控件标识').getElement()
```

示例：

```
let dom = this.$('textfield1').getElement()
dom.style.height = 100
```

wait

用于控件懒加载时，异步获取组件对应DOM对象
格式：

```
this.$('控件标识').wait().then(ctrlDomEle => {})
```

示例：

```
this.$('textfield1').wait().then(ctrlDomEle =>
console.log(ctrlDomEle.style.height)
)
```

CSS

设置控件内联样式，支持修改单个或多个html样式属性
格式：

```
this.$('控件标识').css({'css属性名称':'属性值'})
```

示例：

```
this.$('textfield1').css('background','red')
this.$('textfield1').css({'background':'red', 'color':'#666666'})
```

5、工具类函数

loadFiles

加载资源文件，动态加载第三方资源文件（比如js、css等）
格式：

```
this.utils.loadFiles(['文件路径1','文件路径2']).then(result => {})
```

示例：

```
this.utils.loadFiles(['https://www.xxx.com/xxx/123.js','https://www.xxx.com/xxx/456.css']).then(result => {
console.log(result)
})
```

showMessage

显示消息提示
用于弹出消息提示，接受两个参数：消息内容和可选的配置对象。配置对象中包含type（消息类型，可选值为0-成功，1-错误，2-警告）和duration（显示时长/ms）
格式：

```
this.utils.showMessage('消息内容')
```

示例：

```
this.utils.showMessage('content',{type:0,duration:3000})
```

createStyle

创建页面样式
动态创建style元素并插入到浏览器页面中，当前表单销毁时自动移除
格式：

```
this.utils.createStyle('css样式内容')
```

示例：

```
this.utils.createStyle(".root{width: 10px;} .sel{display: flex}")
```

loadArtTemplate

基于模版创建DOM元素
导入art模版库,方便创建dom元素
格式：

```
this.utils.loadArtTemplate().then(template => {})
```

示例：

```
this.utils.loadArtTemplate().then(template => {
    const id='testSelect', title='this is test art template'
    const str=`<div class='select'><label for='id'>title</label>
<select name="pets" id='id'>{{each items}}<option value=
{{$value.id}}>{{$value.caption}}</option>{{ /each}}</select></div>`
    const items = [{id:1,caption:'name1'},{id:2,caption:'name2'}]
    const htmlString = template.render(str,{items})
    document.getElementById('select').innerHTML = htmlString
})
```

6、其他方法

debugger

调试代码
在js脚本中设置断点，从而支持打开浏览器控制台后调试代码
示例：

```
debugger
```

console

输出日志
在浏览器控制台输出日志信息
示例：

```
console.log('日志信息')
```

六、DOM相关操作

页面脚本目前支持标准的事件（比如字段值改变事件等，后续也会开放更多），数据获取与赋值等操作也有标准的功能接口（比如getValue、setValue等），通过标准事件与功能接口可以快速实现相关业务逻辑。
有些页面交互行为或数据是标准组件/接口没有对外提供或暂未规划的，此时通过组件相关DOM对象的相关操作可以变相的快速实现。
比如：

- 通过DOM对象的'input'事件可以实时监听字段值改变，事件触发时机是实时的（标准字段是失去焦点时才触发）
- 通过DOM对象的'click'事件监听标准按钮的点击事件，在前端页面通过脚本实现相

应业务逻辑

DOM对象事件操作可能对标准组件存在影响，请谨慎使用。
DOM对象监听的事件为HTML DOM原生事件，具体可参考DOM原生事件。
DOM相关操作内置的API包括：on、off、getElement、wait、css，具体描述见"**API介绍**"章节。

七、如何引用第三方JS、CSS等资源

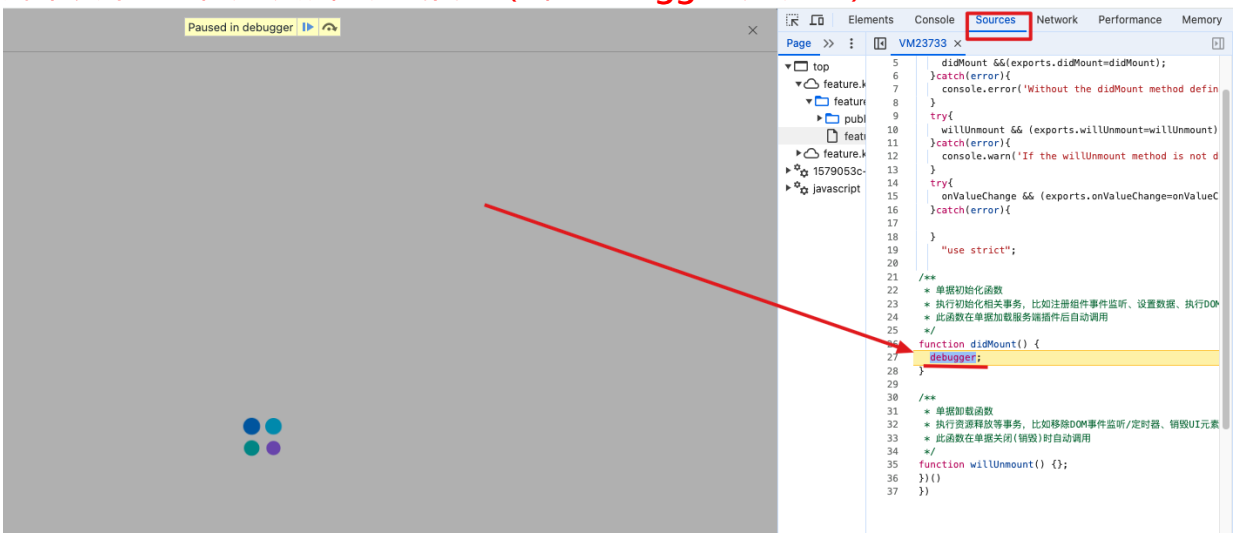
可通过加载第三方资源的API：'**this.utils.loadFiles**' 实现（**请保证自己引用的 JS等资源文件的安全性**）。
示例：

```
1.  // 初始化
2.  function didMount() {
3.   // 加载资源
4.
this.utils.loadFiles(['https://www.xxx.com/xxx/123.js','https://www.xxx.com/xxx/456.css']).then(result=> {
5.   console.log(result)
6.   })
7. }
```

八、如何调试与异常处理

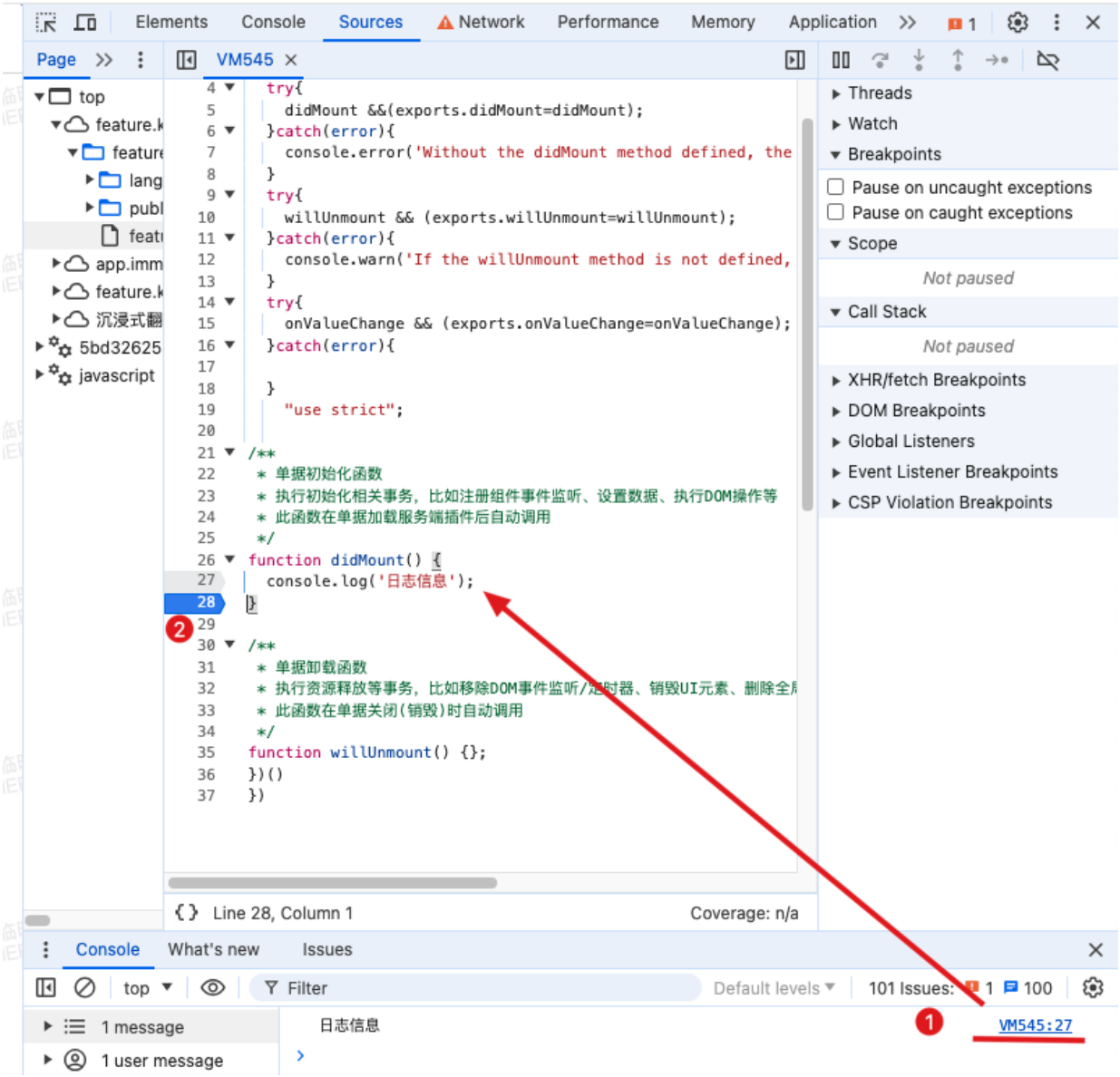
脚本代码调试

1、在脚本中使用"**debugger**"关键字
运行时打开浏览器控制台/开发者工具，即可命中该断点，进入脚本调试
脚本发布上线前建议删除调试代码（即"**debugger**"关键字）

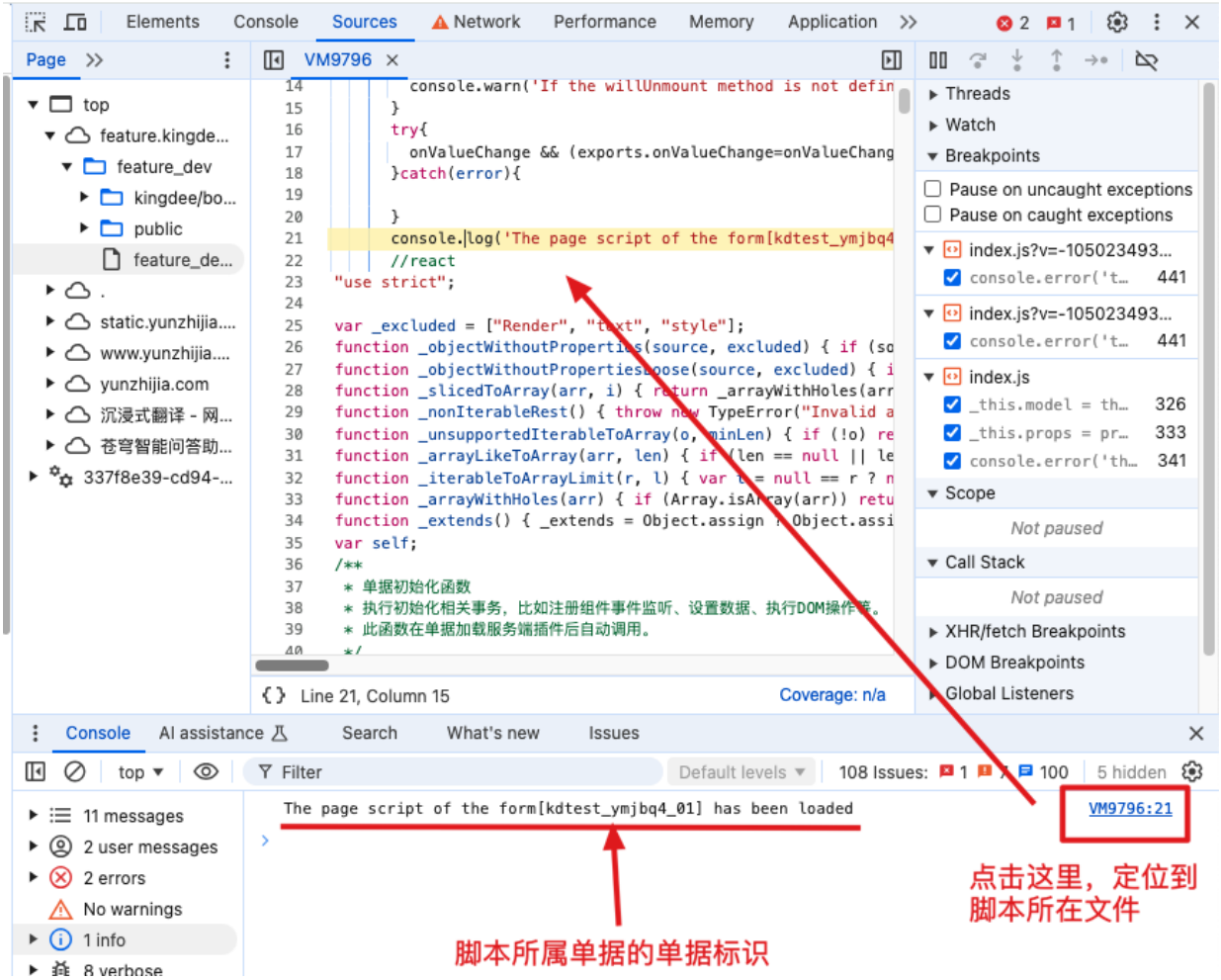


2、通过输出日志来反向定位脚本并打断点
在脚本中任意输出一条日志，比如使用'console.log(日志信息)'，运行时输出的该日志会有对应文档链接，
点击控制台输出的日志其最右边的链接，浏览器会自动打开对应脚本内容，此时再手

动打断点，下次脚本运行到这里就会进入调试。
示例如下：



3、支持开启全局变量"show_ps_init"，脚本初始化时输出固定log信息，方便调试
通过**window.show_ps_init = true**开启该属性后，单据页面初始化时如果有脚本内容，则会输出一条固定log信息（包含脚本所属的单据标识），如下截图。



点击这条log信息最右边的文件链接，则会自动定位到该脚本所在的脚本文件，此时就可以根据需要再脚本文件里打断点做调试了。

开启该属性的方式：

方式1、在浏览器控制台中直接执行脚本语句：window.show_ps_init = true

方式2、如果希望该属性在加载首页单据页面时就生效，则可以尝试在全局扩展JS中开启该属性

备注：单据初始化加载之前开启该属性，单据中的脚本才会输出固定log信息

异常处理

1、脚本中调用内置的标准功能接口（比如“this.$('控件标识').接口名称”），有默认做一些异常捕获，比如控件标识不存在，接口不存在等情况，此时浏览器控制台会有异常信息提示，且**后续脚本会继续执行。**

示例如下：

```
17:08:21.397 ▸ "this.$('kdtest_textfield').badApi()"无法运行；方法"badApi"不存在!!!
17:08:21.400 ▸ "this.$(aaa).set(name,...)"无法运行;控件"aaa"不存在!!!
17:08:21.407 ▸ "this.$(bbb).setValue(...)"无法运行;字段"bbb"不存在!!!
17:08:21.410 ▸ "this.$('ccc').badApi()"无法运行；方法"badApi"不存在!!!
17:08:21.414 ▸ "this.$(kdtest_edit_value).setValue(...)"无法运行;字段"kdtest_edit_value"必须是字段!!!
>
```

2、当用户自己编写的脚本内容引发了异常（比如访问一个不存在对象的某个属性，例如访问a.b但对象a不存在），同时**没有捕获异常时，后续脚本执行就会终止。**

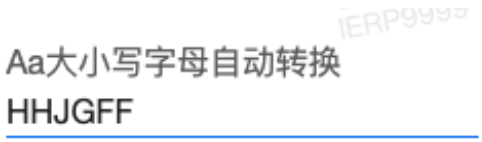
九、案例展示

1、大小写字母实时自动转换

【功能场景】：需要将用户录入的小写英文字母自动并实时转为大写英文字母
【实现思路】：在字段中输入框的原生input事件中做转换，再重新赋值

```
 // 初始化
 function didMount() {
  // 'kdtest_textfield'：文本字段标识；
  this.$('kdtest_textfield').on('input', 'input', (e) => {
  // 给字段赋值
  setTimeout(()=>{
    // setTimeout一下，键盘录入时字母变为大写会有动态过程

this.$('kdtest_textfield').setValue(convertToUpperCase(e.target.valu
e))
  },100)
  })
 }

/**
 * 字母转换为大写
 */
function convertToUpperCase(str) {
  return str.toUpperCase();
}
```

【效果截图】



2、阿拉伯数字转中文数字

【功能场景】：需要将用户录入的阿拉伯数字自动转为中文数字
【实现思路】：在字段值改变的标准事件中做数字转换，再重新赋值

```
  // 初始化
 function didMount() {
  // kdtest_textfield3：文本字段标识；
  this.$('kdtest_textfield3').onValueChange((data) => {
    let result = convertToChineseNumber(data.newValue)
    this.$('kdtest_textfield3').setValue(result)
  })
 }

/**
 * 将字符串中数字转换为大写数字
 */
function convertToChineseNumber(str) {
  // 正则表达式匹配字符串中的所有数字
```

```
  const regex = /\d+/g;
  // 替换函数，将匹配到的数字转换为中文数字
  return str.replace(regex, (match) => {
    const num = parseInt(match, 10); // 将匹配到的字符串转换为数字
    const chineseNumbers = ['零', '壹', '贰', '叁', '肆', '伍', '陆', '柒', '捌', '玖'];
    // 将数字转换为中文数字字符串
    let chineseStr = '';
    for (let i = 0; i < num.toString().length; i++) {
      chineseStr += chineseNumbers[parseInt(num.toString()[i], 10)];
    }
    return chineseStr;
  });
}
```

【效果截图】

3、字段联动，比如做两个数字合计等

【功能场景】：根据页面上某些字段的数据，做二次的合计功能（比如求和或更复杂操作等）

【实现思路】：值改变时做合计，再给显示合计结果的字段赋值

```
  // 单据初始化函数
 function didMount() {
  // 合计值1-值改变事件；kdtest_integerfield1：整数字段标识；
  this.$('kdtest_integerfield1').onValueChange((data) => {
    // 求和，给合计字段赋值
    this.$('kdtest_integerfield').setValue(parseInt(data.newValue) + parseInt(this.$('kdtest_integerfield2').getValue()))
  })
  // 合计值2-值改变事件；kdtest_integerfield2：整数字段标识；
  this.$('kdtest_integerfield2').onValueChange((data) => {
    //求和，给合计字段赋值
    this.$('kdtest_integerfield').setValue(parseInt(data.newValue) + parseInt(this.$('kdtest_integerfield1').getValue()))
  })
 }
```

【效果截图】

4、根据字段值变化做样式修改

【功能场景】：根据页面上某个组件的数据变化，动态修改其他组件的样式，比如实

现一个下拉字段值变化时动态修改自身文字颜色
【实现思路】：在下拉字段切换值触发值改变事件的回调中，修改其输入框前景色

```
  // 单据初始化函数
 function didMount() {
 // kdtest_combofield：下拉字段标识；
  this.$('kdtest_combofield').onValueChange((data) => {
    let colorValue //下拉字段选择的颜色
    switch (data.newValue) {
      case 'red':
        colorValue = 'red'
        break;
      case 'yellow':
        colorValue = 'yellow'
        break;
      case 'blue':
        colorValue = 'blue'
        break;
      default:
        window.alert("未处理的值： " + data.newValue)
    }
    const selectEle = this.$('kdtest_combofield').getElement()
    if (selectEle) {
      // 这里用到一个全局开放CSS: "kd-cq-combo-selected"，用于快速定位输入框文字
      const inputElement = selectEle.querySelector('.kd-cq-combo-selected')
      if (inputElement) {
        // 设置前景色（文字颜色）
        inputElement.style.color = colorValue
      }
    }
  })
  }
```

【效果截图】

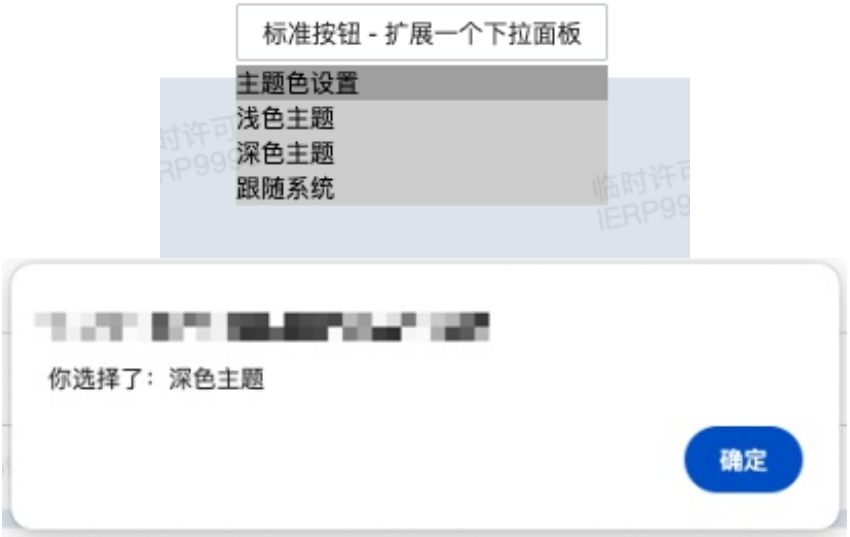
5、标准按钮增加下拉面板

【功能场景】：想扩展现有标准组件实现UI功能定制开发，比如基于标准按钮在点击时实现一个下拉面板并支持点击面板上选项
【实现思路】：通过on接口监听按钮原生click事件，在回调中动态生成下拉面板并且通过on接口监听下拉面板的下拉选项点击事件

```
  // 单据初始化函数
 function didMount() {
  // kdtest_buttonap3：按钮标识；
```

```
  this.$('kdtest_buttonap3').on('click', (e) => {
    const parentDiv = this.$('kdtest_buttonap3').getElement()
    // 防止重复处理
    if (parentDiv && !document.getElementsByClassName('__myChildDivClass__')[0]) {
      // 创建新的子div元素
      let childDiv = document.createElement('div');
      // 设置子div的样式类
      childDiv.className = '__myChildDivClass__';
      // 设置子div的id值、显示位置
      childDiv.id = '__myChildDivClass__';
      let parentPostion = parentDiv.getBoundingClientRect()
      childDiv.style.left = parentPostion.left + "px";
      childDiv.style.top = parentPostion.top + parentPostion.height + 2 + "px";
      childDiv.style.width = parentPostion.width + "px";
      // 将子div添加到父div内
      parentDiv.appendChild(childDiv);
      // 增加下拉面板样式
      this.utils.createStyle('.__myChildDivClass__{background: lightgray;position:fixed;z-index: 1;} .themeselect_div{display: flex;flex-direction: column;color:black;} .themeselect_title{background: darkgray;}')
      //创建下拉面板内容
      this.utils.loadArtTemplate().then(template => {
        const id = 'themeSetting', title = '主题色设置'
        const str = `<div class='themeselect_div' >
                <div class='themeselect_title'>${title}</div>
                <ul name="items" id=${id}>
                    {{each items}}
                        <li value={{$value.id}}>{{$value.caption}} </option>
                    {{ /each}}
                </ul>
            </div>     `
        const items = [{ id: 1, caption: '浅色主题' }, { id: 2, caption: '深色主题' }, { id: 3, caption: '跟随系统' }]
        const htmlString = template.render(str, { items })
        // 将下拉面板内容给到子div渲染
        childDiv.innerHTML = htmlString
        // 绑定处理下拉项点击事件
        this.$('kdtest_buttonap3').on('click','li', (e)=>{
          alert('你选择了：' + e.target.outerText);
        })
      })
    }
  })
}
```

【效果截图】



6、自定义按钮点击

【功能场景】：业务或二开需要在前端页面上某个标准按钮被点击时，做一些自定义的页面逻辑实现，不依赖服务端插件
【实现思路】：通过原生click事件监听按钮点击（后续标准事件也可能支持）

```
  // 单据初始化函数
 function didMount() {
  // 'kdtest_buttonap1'：按钮标识；【实现思路】：通过原生click事件监听按钮点击（后续标准事件也可能支持）
  this.$('kdtest_buttonap1').on('click', (e) => {
    window.alert('你点击了按钮');
  })
 }
```

7、标准字段（页面脚本）与自定义控件通信

【通信方向】页面脚本 => 自定义控件内部发送消息
【功能场景】：开发的自定义控件其渲染依赖页面上某些字段的数据，所以需要在这些字段数据有变化时通知到自定义控件内部，从而刷新自定义控件内部数据并重新渲染，不依赖服务端插件
【实现思路】：
1、监听字段（这里以文本字段为例）值变化事件，在事件回调中调用控件通用消息接口 ‘**inovke**’ 向自定义控件发送消息
2、自定义控件内部接收页面JS脚本消息：在自定义控件内部（入口文件index.js）已有方法'**handleDirective**'中处理（方法介绍）
JS脚本代码示例：

```
  // 单据初始化函数
 function didMount() {
  // 字段值变化时向自定义控件内部发送消息
  //'kdtest_textfield'：字段组件标识
  this.$('kdtest_textfield').onValueChange((data) => {
      //'customcontrolap'：要接收消息的自定义控件标识；invoke接口参数说明：方法名称(自定义)、方法参数(自定义)
     this.$('customcontrolap').invoke('textFieldValueChange01',{a1:
```

```
    '1121', data: data.newValue});
  })
}
```

自定义控件内部代码示例（index.js）：

```
  //自定义控件内部接收页面JS脚本消息：在自定义控件内部已有方法handleDirective中处理
  handleDirective: function (customProps, methodname, arg){
    //按照上面JS脚本示例调用脚本，此时该方法参数值为：methdoname=>'textFieldValueChange01' arg=>{a1: '1121', data: data.Value}
    if(methodname == 'textFieldValueChange01'){
      alert(arg.data);
    }
  }
```

【通信方向】自定义控件内部 => 页面脚本中发送消息
【功能场景】：需要将自定义控件内部一些数据发送到页面脚本中使用
【实现思路】：
1、在设计器中给自定义控件绑定"**自定义消息订阅**"事件，从而监听自定义控件内部发出来的消息
2、在自定义控件内部调用标准消息接口“**model.triggerCustomMsgEvent**”将自定义控件内部的数据传送到页面脚本中（数据在步骤1中事件里接收，即调用接口会触发事件）
JS脚本代码示例：

```
  // 单据初始化函数
 function didMount() {
    //页面脚本中事件绑定代码
    this.$('customcontrolAp').onCustomMsgEvent((data) => {
        //自定义控件内部发送的信息就在参数data中
    })
 }
```

自定义控件内部代码示例（index.js）：

```
  //在需要向页面脚本中发送消息的地方调用如下代码
  //调用示例（接口封装在model对象中，注意上下文）
  this.model.triggerCustomMsgEvent('dataChange', {name: '名称1', code: '888'}); //调用：接口方式一
  this.model.triggerCustomMsgEvent({type: 'dataChange', name: '名称1', code: '888'}); //调用：接口方式二
```

8、页面脚本与服务端插件通信

【功能场景】：在前端页面扩展脚本中需要向服务端通信去获取特定的业务数据，比如扩展了一个按钮，点击该按钮需要与服务端通信并执行特定的业务逻辑。
【实现思路】：
1、前端页面直接调用fetchData API接口发送网络请求，服务端插件通过重载自定义事件customEvent来接收请求消息（插件即为脚本所在页面的插件）。

2、发送到服务端请求使用了固定自定义事件key："__clientRequest__"，以此可以判断是不是页面脚本的请求
3、在服务端插件中通过全局指令"setPageJSData"将服务端数据返回到前端页面脚本中
前端页面JS脚本代码示例：

```
//发送网络请求
this.fetchData('getUserInfo', {userName:'xiaoming', userCode:'123'}).then((result)=>{
})
```

服务端插件示例（**以下为服务端KS脚本案例，java插件写法可自行转换**）：

```
//服务端插件，重写自定义事件
customEvent(e: CustomEventArgs) {
    const key = e.getKey()
    const eventName = e.getEventName() //自定义事件：事件名称
    const eventArgs = e.getEventArgs() //自定义事件：事件参数
    //页面脚本fetchData请求对应自定义事件key为固定的："__clientRequest__"
    if (key === '__clientRequest__') {
        if (eventName === 'getUserInfo') {
            //通过全局指令"setPageJSData"将服务端数据返回到前端页面脚本中

this.getView().getClientProxy().addAction('setPageJSData', { 'name': 'test01', 'args': { 'a1': '222' } })
        }
    }
}
```

9、树控件节点自定义渲染或扩展

需求描述：支持在标准树节点上扩展新的UI元素，以实现一直或鼠标hover时显示自定义节点元素及与服务端交互
使用场景：业务在树节点上扩展一个“新增”按钮，点击该按钮实现业务自定义逻辑，比如在特定位置新增一个树节点等（一般点击按钮都需要与服务端插件交互）
案例代码：
功能说明：在树节点上扩展一个按钮出来，点击与服务端交互。

```
let self
/**
 * 单据初始化函数
 * 执行初始化相关事务，比如注册组件事件监听、设置数据、执行DOM操作等。
 * 此函数在单据加载服务端插件后自动调用。
 */
function didMount() {
  self = this
  // 设置样式，控制树节点元素显示与隐藏。'kd-cq-tree-treenode'是树控件节点山的一个全局样式名称
```

```
  this.utils.createStyle(`
    .kd-cq-tree-treenode:hover .treeNodeExt{
      visibility:visible!important;
    }
  `)
      //扩展树节点
      this.$('kdtest_treeviewap').setTreeItemRender(TreeItemRender)
}

/**
 * 树控件，节点扩展（渲染自定义）
 * 参数'props'内容: Render/渲染器、text/树节点名称、...
 */
function TreeItemRender(props) {
  const { Render, text, style = {}, ...others } = props
  // 扩展元素点击
  const onLinkClick = event => {
    event.preventDefault()
    //向服务端发送请求
    const nodeId = event.currentTarget.getAttribute('data-treemenuid')
    self.fetchData('treeMenuClick', { 'treeMenuId': event }).then((data) => {
      alert("treeMenuClick: " + nodeId);
    })
  }

  // 扩展元素内容
  const extMenu = <a
    onClick={onLinkClick}
    href="true"
    className="kd-dropdown-link treeNodeExt"
    style={{
      width: '100px',
      display: 'flex',
      flexGrow: 1,
      visibility: 'hidden'
    }}
    data-treeMenuId={props.id}
  >
    <span style={{ whiteSpace: 'nowrap' }}>新增({props.id})</span>
  </a>
  //Slot属性用于传递扩展UI对象、text属性代表树节点名称
  return <Render {...others} text={text + '_ext'} Slot={extMenu} />
}
```

案例截图：

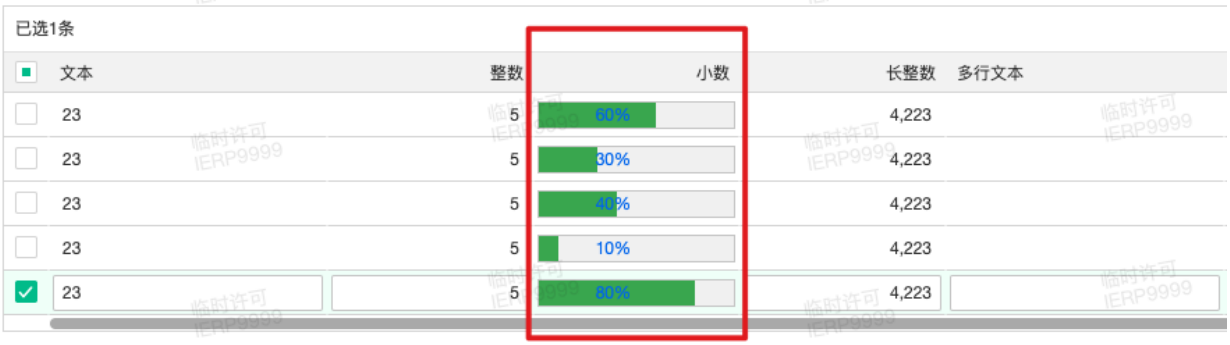
10、表格单元格自定义渲染或扩展

需求描述：支持单元格渲染自定义（查看态和编辑态），定制UI外观，交互事件等
使用场景：
1、【查看态】根据业务需要和表格数据，自定义UI外观。比如将数据为小数类型的单元格（数字类型字段）其外观自定义显示为进度条效果
2、【编辑态】根据业务需要自定义单元格编辑器
查看态 -【重新渲染整个单元格】
功能说明：将值为小数类型单元格其查看态UI外观定制为进度条效果。
案例代码（查看态）

```
/**
 * 单据初始化函数
 * 执行初始化相关事务，比如注册组件事件监听、设置数据、执行DOM操作等。
 * 此函数在单据加载服务端插件后自动调用。
 */
function didMount() {
  //设置单元格渲染器('kdtest_entryentity'：表格标识、'kdtest_decimalfield':表格某小数类型列标识)
  this.$('kdtest_entryentity').setCellRender({
  kdtest_decimalfield: progressRender
  })
}

//进度条效果
const progressRender = props => {
  const { value } = props
  //进度条父容器
  const progressContainer = {
    width: '100%',
    height: '20px',
    'background-color': '#f2f2f2',
    border: '1px solid #ccc',
    position: 'relative'
  }
```

```
  const barWidth = value * 100 + '%'
  //进度条的进度
  const progressBar = {
    height: '100%',
    'background-color': '#4CAF50',
    width: barWidth
  }
  const progressLabel = {
    position: 'absolute',
    top: 0,
    right: '50%',
    padding: '2px 4px',
    color: '#276ff5'
  }
  //返回渲染对象
  return (<div style={progressContainer}>
    <div style={progressBar}></div>
    <div style={progressLabel}>{Math.round(value * 100) + '%'}</div>
  </div>)
}
```

案例截图（查看态）



查看态 -【基于现有单元格渲染扩展新的UI元素】

功能说明： 在偶数行的单元格上扩展一个图标，以及修改现有单元格元素的样式（背景色）。

案例代码（查看态）

```
/**
 * 单据初始化函数
 * 执行初始化相关事务，比如注册组件事件监听、设置数据、执行DOM操作等。
 * 此函数在单据加载服务端插件后自动调用。
 */
function didMount() {
    //设置单元格渲染器（特定列）
    //'kdtest_entryentity'表格标识、'kdtest_bigintfield'列标识
    this.$('kdtest_entryentity').setCellRender({
      kdtest_bigintfield: extendIconRender
    })
}

/**
 * 表格的列查看态，基于现有渲染，扩展UI元素
```

```
 */
const extendIconRender = props => {
  const { value, record, rowIndex, renderProps, parentElement, Render, isCellLock } = props
   //“value”属性为格式化后的值（用于在单元格中显示，非原始值；原始值要从props中的属性“originValue”获取），处理时请谨慎
  const cellValue = Array.isArray(value) ? value[0].text : value
  const cellText = cellValue || `row=${rowIndex},col=${renderProps.column.code}`
  //修改默认渲染元素样式
  if (rowIndex % 2 === 1) { //偶数行
    style.backgroundColor = 'blue'
    parentElement.style.backgroundColor = 'yellow'
  } else { //奇数行
    style.backgroundColor = 'red'
    parentElement.style.backgroundColor = 'red'
  }

  //偶数行才扩展一个图标出来（根据需要可以再给图标添加交互事件等）
  if (rowIndex % 2 === 1) {
    //扩展图标元素样式
    let iconStyle = {
      position: 'absolute',
      right: 0,
      top: '5px',
      zIndex: 1
    }
    //这里用到了字体图标，也可使用img元素实现
    return (
      <>
        <i style={iconStyle} className='kdfont kdfont-tishixinxi_l' />
        <Render {...{ rowIndex, renderProps, record, value: cellText }} />
      </>
    )
  }
  return <Render {...{ rowIndex, renderProps, record, value: cellText }} />
}
```

案例截图（查看态）

编辑态 -【重新渲染整个单元格】

功能说明：将下拉类型字段单元格编辑器自定义为'性别选择'的平铺选项效果

案例代码（编辑态）

```

/**
 * 单据初始化函数
 * 执行初始化相关事务，比如注册组件事件监听、设置数据、执行DOM操作等。
 * 此函数在单据加载服务端插件后自动调用。
 */
function didMount() {
    //设置单元格编辑器('kdtest_entryentity'：表格标识、'kdtest_sexy':表格某下拉字段类型列标识)
    this.$('kdtest_entryentity').setCellEditor({
      'kdtest_sexy': SexyEditor
    })
}

//编辑态 - 单元格渲染
const SexyEditor = props => {
  const { value, updateEditValue } = props
  const [sexy, setSexy] = useState(value)
  const style = {
    display: 'flex',
    justifyContent: 'space-evenly',
    width: '100%',
    height: 18
  }
  const labelStyle = {
    height: 'initial',
    whiteSpace: 'nowrap'
  }
  const inputStyle = {
    height: 'initial',
```

```
    marginLeft: 2
  }
  const onClick = event => {
    const target = event.target
    if (target.tagName === 'INPUT') {
      setSexy(target.dataset.value)
      updateEditValue(parseInt(target.dataset.value))
    }
  }
  //react组件写法（性别选择）
  return (<div style={style} onClick={onClick}>
    <label style={labelStyle}>男<input style={inputStyle} type='radio' data-value={0} checked={sexy == 0} /></label>
    <label style={labelStyle}>女<input style={inputStyle} type='radio' data-value={1} checked={sexy == 1} /></label>
    <label style={labelStyle}>保密<input style={inputStyle} type='radio' data-value={2} checked={sexy == 2} /></label>
  </div>)
}
```

案例截图（编辑态）

编辑态 -【基于现有单元格渲染扩展新的UI元素】
功能说明：基于现有的编辑态单元格渲染，修改其样式（也可以自己扩展UI元素）
案例代码（编辑态）

```

/**
 * 单据初始化函数
 * 执行初始化相关事务，比如注册组件事件监听、设置数据、执行DOM操作等。
 * 此函数在单据加载服务端插件后自动调用。
 */
function didMount() {
    //设置单元格编辑器('kdtest_entryentity'：表格标识、'kdtest_timerangefield':表格某列标识)
    this.$('kdtest_entryentity').setCellEditor({
      'kdtest_timerangefield': extendCellEditor
    })
}

```

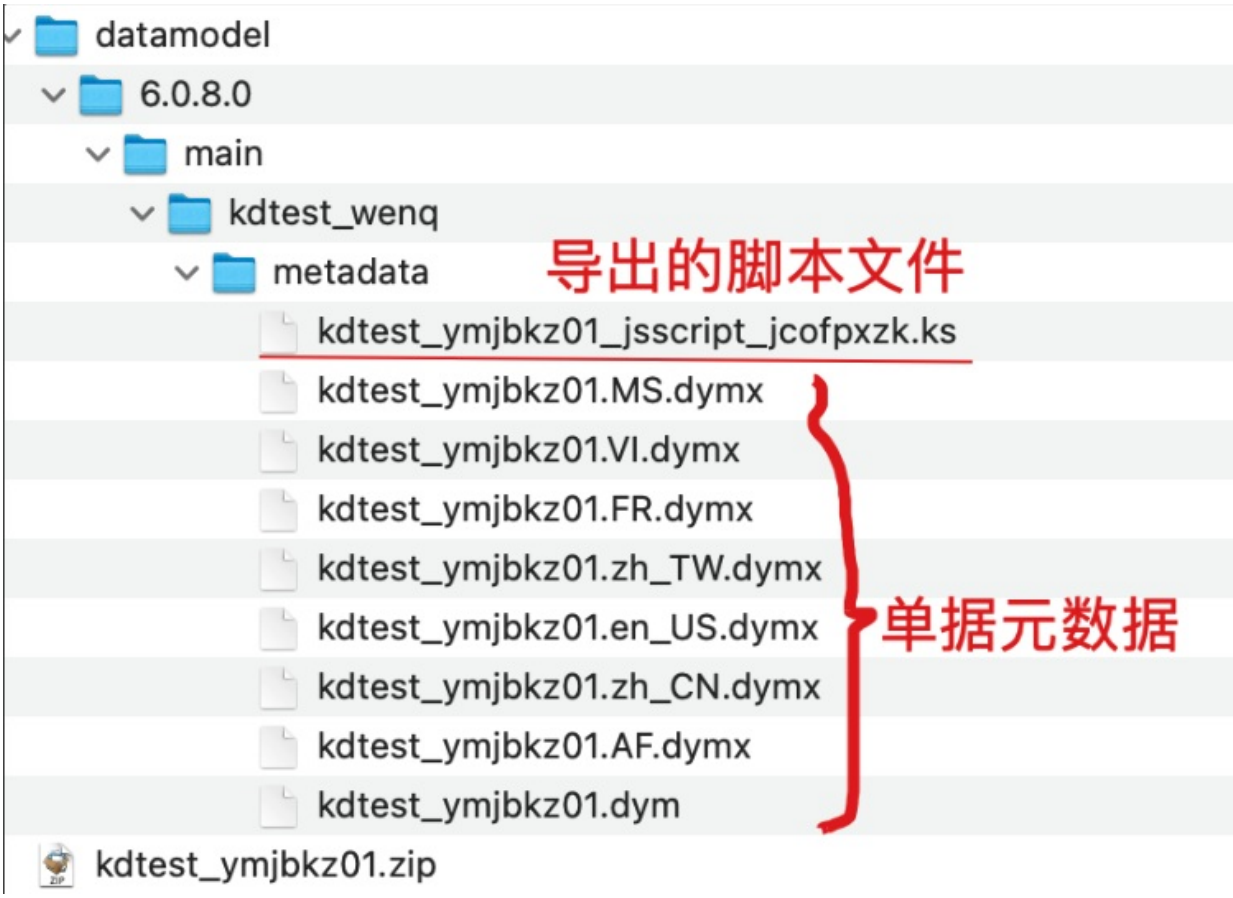
```
/**
 * 表格的列编辑态，基于现有渲染，修改样式或扩展UI元素
 */
const extendCellEditor = props => {
  const { Editor, renderProps, value, record, rowIndex, updateEditValue } = props
  useEffect(() => {
    const ele = props.getContainerEle()
    const input = ele.querySelector('input')
    input.style.fontSize = '24px'
    input.style.backgroundColor = 'gray'
    // 提示信息
    input.placeholder = `请选择日期范围`
  }, [])

  const _updateEditValue = value => {
    return updateEditValue(value.charAt(0).toUpperCase() + value.slice(1))
  }
  return <Editor {...props} updateEditValue={_updateEditValue} />
}
```

案例截图（编辑态）

十、脚本部署与迁移(导入导出)

脚本发补丁

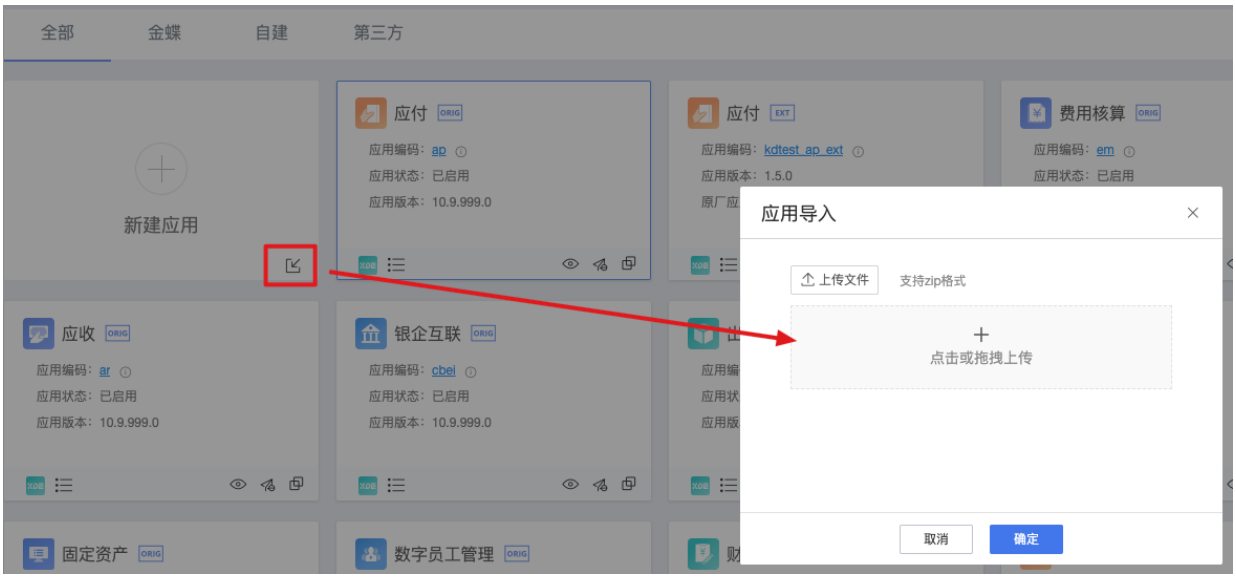
页面脚本在服务端是独立的数据库表存储，脚本与单据的关联关系数据是存储在单据元数据中的。
如果是制作标准补丁，则在开发平台导出页面数据（**此时页面脚本和页面元数据会一起导出，页面脚本文件扩展名'.ks'**）再提交到对应元数据目录即可。



脚本迁移(导入导出)

如何将页面脚本从一个环境迁移到另一个环境？
答：基于开发平台的导入导出功能，将单据和对应页面脚本从一个环境导出，再导入另一个环境即可
【开发平台 - 导出】

【开发平台 - 导入】



注意：在开发平台手工导入页面（包含页面脚本），如果页面有依赖其他单据比如父页面等，需要先导入父页面，否则导入会有异常，脚本内容也会导入失败。

十一、注意事项

1、IE浏览器暂不支持脚本编辑，可执行脚本。
2、'set'接口设置属性可能会被服务端插件设置内容覆盖（原因是服务端插件修改生效优先级>=页面脚本）。
3、脚本执行异常（未捕获的），则后续脚本执行中断。
4、通过'on'接口监听组件DOM事件，该事件不会与组件内部状态联动。比如锁定的按钮，通过on接口监听其'click'点击事件，即使按钮通过被锁定了，事件依然会触发。
5、**'set'接口支持的参数（控件属性）有限，一般都是与样式设置相关。**参数获取示例：修改控件宽度，其参数获取方法（设计器中鼠标放到对应属性名称上即可查看到参数名称“Width”），如下截图：

备注：为保证已有业务逻辑的正确性，锁定性、可见性、必录、控件标识、类型等控件属性不支持通过脚本修改

6、'**createStyle**'接口创建Class限制其作用域在当前单据页面(防止全局样式污染)。

7、使用'**setValue**'接口为基础资料类型字段赋值会引发服务端报错，原因是基础资料字段赋值是要在服务端先赋值，再刷新前端组件（这与文本字段等不同）。

8、**历史单据新增页面脚本，脚本保存之后需要再做一次设计器的保存，否则脚本内容会丢失。**

9、脚本中标准事件或其他API接口（比如on）中使用的回调函数建议使用箭头函数写法，可自动绑定上下文，保证在函数中可正常访问脚本内置对象等。

十二、常见问题与解决方法

1、怎样获取控件Dom对象？

【解决办法】：使用页面脚本中提供的标准接口getElement或htlm的一些原生接口都可以实现

【代码示例】：

```
1. //【推荐】通过页面脚本标准接口（获取整个控件的dom对象）
2. const domEle = this.$('控件标识').getElement() //同步
3. this.$('控件标识').wait().then(ctrlDomEle => { }) //异步
4. 或者
```

```
//通过html原生接口（根据页面上页面上控件对应Dom结构里的'id'属性获取，一般就是控件标识）
const element = document.getElementById('元素ID');
或者
//其他可参考的原生接口如下
const elements = document.getElementsByClassName('类名'); //根据class类名
const elements = document.getElementsByTagName('标签名'); //根据元素类型
const element = document.querySelector('CSS选择器'); //获取第一个
const elements = document.querySelectorAll('CSS选择器'); //获取多个
```

2、在didMount里获取控件Dom对象是空的？

【原因分析】：
1、控件布局在容器中，但容器开启了懒加载，导致页面初始化时容器中控件可能没有立即渲染，就没有对应Dom信息。
2、控件较复杂，初始化耗时长一些导致其挂在到页面Dom结构上慢一些，此时获取不到对应Dom对象
【解决办法】：
1、**【推荐】**在单据对应表单设计器中，使用'wait'接口异步获取控件Dom对象。
示例代码：

```
//ctrlDomEle即为控件对应的Dom对象
this.$('控件标识').wait().then(ctrlDomEle => { })
```

2、复杂控件如表格和树等，都有对外提供一个“控件初始化”事件，可在该事件中获取其Dom对象
3、如果是其他场景无法获取到控件对应Dom对象，可以尝试在页面脚本didMount中使用setTimeout延迟获取，参考示例代码如下

```
//在页面初始化时延迟300ms获取dom对象
setTimeout(() => {
  cont domEle = this.$('控件标识').getElement()
  }, 300);
```

3、怎样监听原生html事件？

【解决办法】：使用页面脚本中提供的标准接口'on'或htlm的一些原生接口都可以实现
【代码示例】：

```
//【推荐】通过页面脚本标准接口（直接监听整个控件的某些事件，或者控件内部某个子元素的事件）
this.$('控件标识').on('click', (e)=>{ })
或者
//通过html原生接口（先获取要监听目标对应dom对象，监听对应事件）
cont domEle = this.$('控件标识').getElement()
domEle.addEventListener('click', function(event) { })
```

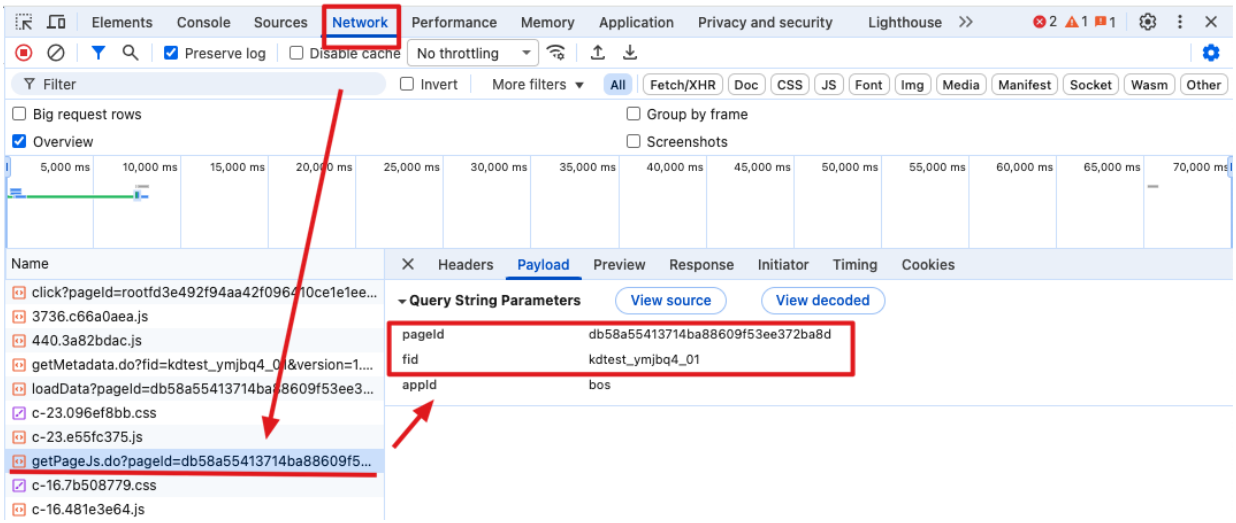
4、怎样通过页面脚本修改页面内特定控件的样式？

【解决办法】：使用页面脚本标准接口'css'或'createStyle'
【代码示例】：

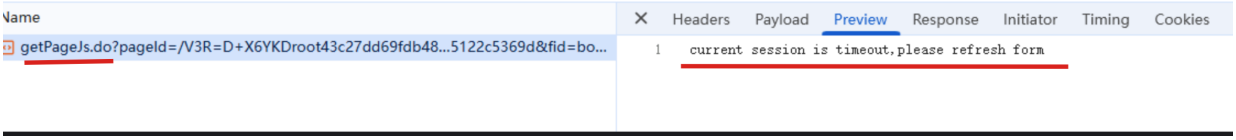
```
1. //css接口（内容格式跟原生css一致），修改标识为'kdtest_beizhu'的控件样式
2. this.$('kdtest_beizhu').css({ 'color': 'yellow' })
3. 或者
4. //createStyle接口（该接口作用范围为当前单据页面），修改标识
   为'kdtest_beizhu'的字段上标题的样式
5. this.utils.createStyle('[id=kdtest_beizhu] .kd-cq-field-title-
   wrap{font-Size:30px !important}')
```

5、怎样排查页面js脚本在单据页面上不生效的问题？

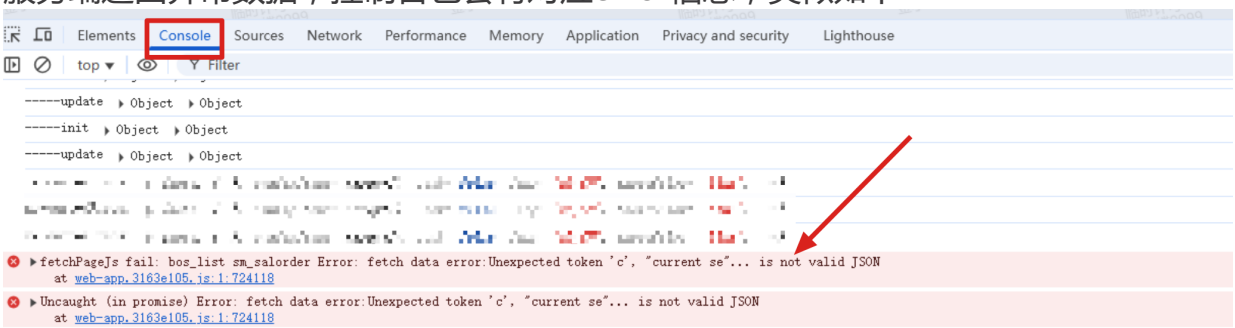
【原因分析】
问题1、页面没有加载JS脚本或内容加载异常
问题2、页面加载的JS脚本内容不是最新的
问题3、编写的JS脚本内容与浏览器或具体业务数据相关，运行时获取到相关对象不存在导致程序执行失败
【解决办法】
问题1、通过脚本内容接口请求，确认是否加载以及加载脚本内容是否异常
通过浏览器控制台查看加载JS脚本请求接口"getPageJs.do"（带有页面标识等信息）：
前端页面会缓存已加载脚本数据，如果脚本内容无变化，下次打开单据不会再发送该接口请求（后续脚本内容有变化或清理掉浏览器缓存，再打开单据就会发送该接口请求）。



加载脚本内容异常示例如下：
左侧为通过浏览器控制台查看的单据页面请求对应页面JS脚本的数据，请求接口“getPageJs.do”、
右侧为接口请求返回的异常数据（正常数据应该包含js脚本等内容，json数据格式）



服务端返回异常数据，控制台也会有对应error信息，类似如下



问题2、脚本内容是否最新需要自己查看加载的脚本内容或直接调试也可
可查看请求的脚本数据内容或通过调试脚本代码，以确认最新改动JS脚本内容是否已加载；
也可以在有改动内容地方输出日志，方便查看
问题3、需要自己调试代码查看
怎样调试代码请查看章节八"脚本代码调试"部分内容。

十三、技术支持与资源

学习资源

JavaScript脚本学习教程

技术支持

建议方式如下：
1、苍穹开发者论坛发帖
2、KSM系统提单
3、需求反馈: qiao_wen@kingdee.com